

CalcScript — Analizador Léxico

Proyecto de Programación de Sistemas de Base 1

Campo	Valor
Materia	Programación de Sistemas de Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor(es)	Dr. Juan Carlos Méndez Ríos
Integrantes	Ana Sofía Ramírez Vega Luis Eduardo Torres Garza Daniela Hernández Leal Carlos Iván Morales Peña
Repositorio	https://github.com/equipo3-compiladores/calcscrip-lexer
Fecha	Junio 2026

El presente documento constituye la Demostración Funcional del proyecto de semestre para la materia de Programación de Sistemas de Base 1. Incluye la especificación del lenguaje diseñado, el código fuente comentado del analizador léxico y capturas de ejecución con casos válidos y con errores.

Tabla de contenido

Introducción	1
Objetivos	2
Objetivo general	2
Objetivos específicos	2
Especificación del Lenguaje	3
3.1 Nombre y propósito	3
3.2 Ejemplos ilustrativos de código	3
3.3 Catálogo de tokens	4
Implementación	5
4.1 Definición de tokens y palabras reservadas — tokens.py	5
4.2 Reglas léxicas — rules.py	6
4.3 Lógica del analizador léxico — lexer.py	6
4.4 Manejo y reporte de errores — main.py	7
Demostración Funcional	9
5.1 Ejecución con programa válido	9
5.2 Ejecución con errores léxicos	10
5.3 Caso con identificadores inválidos y símbolos desconocidos	11
Conclusiones	13
Trabajo futuro (opcional)	13
Referencias	14

Sección 1 — Introducción

El análisis léxico es la primera fase de un compilador o intérprete: recibe una secuencia de caracteres y produce una secuencia de tokens que representan las unidades mínimas con significado del lenguaje. Comprender esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee construir herramientas de procesamiento de lenguajes.

El presente proyecto diseña e implementa un analizador léxico para **CalcScript**, un lenguaje personalizado de propósito educativo orientado a la escritura de programas de cálculo aritmético con soporte para variables, estructuras de control básicas y funciones. El nombre refleja su propósito principal: un lenguaje de *scripting* para cálculos.

El lexer fue implementado en Python 3.12 sin dependencias externas, usando expresiones regulares de la biblioteca estándar (`re`). Se diseñó para transformar archivos de texto fuente `.csc` en una lista numerada de tokens, e informar con precisión la línea y columna de cualquier error léxico detectado.

Este documento se organiza de la siguiente manera: la Sección 2 presenta los objetivos del proyecto; la Sección 3 describe la especificación del lenguaje CalcScript; la Sección 4 contiene el código fuente comentado del lexer; la Sección 5 muestra capturas de ejecución; finalmente las Secciones 6 y 7 presentan conclusiones y referencias.

Sección 2 — Objetivos

Objetivo general

Diseñar un lenguaje personalizado denominado **CalcScript** e implementar un analizador léxico funcional en Python que transforme programas fuente escritos en dicho lenguaje en listas de tokens, con manejo preciso de errores léxicos.

Objetivos específicos

1. Definir la especificación informal del lenguaje CalcScript mediante ejemplos ilustrativos de código que cubran sus construcciones principales.
2. Establecer un catálogo completo de tokens: palabras reservadas, identificadores, números enteros y flotantes, operadores aritméticos y relacionales, delimitadores y cadenas de texto.
3. Implementar el analizador léxico usando expresiones regulares, asegurando el reconocimiento correcto de todos los tokens definidos.
4. Implementar la detección y reporte de errores léxicos, indicando número de línea, columna y descripción del problema para cada error encontrado.
5. Probar el lexer con al menos tres programas válidos y dos programas con errores léxicos que cubran los casos descritos en la especificación.

Sección 3 — Especificación del Lenguaje

3.1 Nombre y propósito

CalcScript es un lenguaje imperativo de propósito educativo diseñado para expresar cálculos aritméticos, asignaciones de variables, estructuras de control (if/else, while) y definición de funciones simples. La extensión de los archivos fuente es `.csc`.

3.2 Ejemplos ilustrativos de código

Ejemplo 1 — Cálculo de factorial:

```
func factorial(n) {
    result = 1;
    while n > 1 {
        result = result * n;
        n = n - 1;
    }
    return result;
}

x = factorial(5);
print(x);
```

Ejemplo 2 — Condicional con flotantes:

```
a = 3.14;
b = 2.0;

if a > b {
    diff = a - b;
    print(diff);
} else {
    print(b);
}
```

Ejemplo 3 — Expresión aritmética simple:

```
total = (10 + 5) * 2 / 3;
print(total);
```

3.3 Catálogo de tokens

Categoría	Ejemplos	Descripción
KEYWORD	if, else, while, func, return, print	Palabras reservadas del lenguaje
IDENTIFIER	x, total, my_var, result2	Letras o <code>_</code> seguidas de letras, dígitos o <code>_</code>
NUMBER_INT	0, 42, 1024	Secuencia de dígitos sin punto decimal
NUMBER_FLOAT	3.14, 0.5, 2.0	Dígitos, punto, dígitos obligatorios
OPERATOR	+, -, *, /, =, >, <, >=, <=, ==, !=	Operadores aritméticos y relacionales
DELIMITER	;, () { }	Separadores y agrupadores

STRING	"hola", "valor"	Cadena entre comillas dobles
COMMENT	# esto es un comentario	Desde # hasta fin de línea (se descarta)

Identificadores válidos: comienzan con letra (a-z, A-Z) o guión bajo (_), seguidos de cero o más letras, dígitos o guiones bajos.

Identificadores inválidos (generan error): comienzan con dígito (123abc), contienen caracteres especiales no permitidos (my - var, x@y).

Sección 4 — Implementación

A continuación se presenta el código fuente de los módulos del analizador léxico. Los fragmentos han sido seleccionados para ilustrar los puntos de mayor relevancia conceptual; el código completo se encuentra en el repositorio GitHub indicado en la portada.

4.1 Definición de tokens y palabras reservadas — tokens.py

Define la enumeración de tipos de token y el conjunto de palabras reservadas. Usar una clase de constantes centralizadas garantiza que lexer y futuras fases del compilador compartan exactamente los mismos identificadores.

```
# tokens.py
# Define los tipos de token reconocidos por CalcScript y las palabras
# reservadas que no pueden usarse como identificadores.
#
# Usar una clase de constantes centralizadas (Enum) garantiza que el lexer
# y futuras fases del compilador compartan exactamente los mismos
# identificadores de tipo, evitando errores de comparación por strings.

from enum import Enum, auto

class TokenType(Enum):
    """Tipos de token reconocidos por el analizador léxico de CalcScript."""

    # Literales
    NUMBER_INT = auto()      # entero: 42, 0, 1024
    NUMBER_FLOAT = auto()    # flotante: 3.14, 0.5
    STRING = auto()          # cadena: "hola"
    IDENTIFIER = auto()      # nombre de variable o función

    # Palabras reservadas (se detectan reclasificando IDENTIFIER)
    KEYWORD = auto()

    # Operadores: + - * / = > < >= <= == !=
    OPERATOR = auto()

    # Delimitadores: ; , ( ) { }
    DELIMITER = auto()

    # Control interno
    EOF = auto()             # fin de archivo
    ERROR = auto()           # token inválido (se reporta y se descarta)

# Conjunto de palabras reservadas. Se usa para reclasificar
# un IDENTIFIER reconocido en un KEYWORD cuando coincide con esta tabla.
KEYWORDS: set[str] = {
    "if", "else", "while", "func", "return", "print",
}
```

4.2 Reglas léxicas — rules.py

Centraliza todas las expresiones regulares del lenguaje. Cada regla es una tupla (TokenType, patron). El orden importa: NUMBER_FLOAT debe preceder a NUMBER_INT para que 3.14 no se

tokenice como 3, ., 14.

[illegible]

4.3 Lógica del analizador léxico — `lexer.py`

Implementa el bucle principal del lexer. En cada iteración intenta hacer coincidir las reglas en orden desde la posición actual; si ninguna regla coincide, emite un error con la ubicación exacta y avanza un carácter para evitar un bucle infinito.

[illegible]

[illegible]


```

except OSError as exc:
    print(f"Error al leer '{source_path}': {exc}")
    sys.exit(1)

lexer = Lexer(source)
tokens, errors = lexer.tokenize()

# ■■ Salida: lista de tokens ■■
print()
print("Token List:")
for tok in tokens:
    print(f" {tok}")

# ■■ Salida: lista de errores (si hay) ■■
print()
if errors:
    print("Error List:")
    for err in errors:
        print(f" {err}")
else:
    print("No lexical errors found.")

if __name__ == "__main__":
    main()

```

Sección 5 — Demostración Funcional

Las siguientes capturas muestran la ejecución del lexer desde la terminal. Cada escenario fue ejecutado con `python src/main.py` en Linux (Ubuntu 24.04, Python 3.12.3).

5.1 Ejecución con programa válido

Se tokeniza el archivo `tests/valid/factorial.csc` que implementa el cálculo de factorial. El programa no contiene errores léxicos.

Archivo de entrada — `tests/valid/factorial.csc`:

```
func factorial(n) {
    result = 1;
    while n > 1 {
        result = result * n;
        n = n - 1;
    }
    return result;
}

x = factorial(5);
print(x);
```

Ejecución y salida:

```
$ python src/main.py tests/valid/factorial.csc
```

Token List:

```
1: (KEYWORD, "func")
2: (IDENTIFIER, "factorial")
3: (DELIMITER, "(")
4: (IDENTIFIER, "n")
5: (DELIMITER, ")")
6: (DELIMITER, "{")
7: (IDENTIFIER, "result")
8: (OPERATOR, "=")
9: (NUMBER_INT, "1")
10: (DELIMITER, ";")
11: (KEYWORD, "while")
12: (IDENTIFIER, "n")
13: (OPERATOR, ">")
14: (NUMBER_INT, "1")
15: (DELIMITER, "{")
16: (IDENTIFIER, "result")
17: (OPERATOR, "=")
18: (IDENTIFIER, "result")
19: (OPERATOR, "*")
20: (IDENTIFIER, "n")
21: (DELIMITER, ";")
22: (IDENTIFIER, "n")
23: (OPERATOR, "=")
24: (IDENTIFIER, "n")
25: (OPERATOR, "-")
26: (NUMBER_INT, "1")
27: (DELIMITER, ";")
28: (DELIMITER, "}")
```

```
29: (KEYWORD, "return")
30: (IDENTIFIER, "result")
31: (DELIMITER, ";")
32: (DELIMITER, "}")
33: (IDENTIFIER, "x")
34: (OPERATOR, "=")
35: (IDENTIFIER, "factorial")
36: (DELIMITER, "(")
37: (NUMBER_INT, "5")
38: (DELIMITER, ")")
39: (DELIMITER, ";")
40: (KEYWORD, "print")
41: (DELIMITER, "(")
42: (IDENTIFIER, "x")
43: (DELIMITER, ")")
44: (DELIMITER, ";")
```

No lexical errors found.

Captura 5.1 — Tokenización completa del programa factorial.csc sin errores.

5.2 Ejecución con errores léxicos

Se tokeniza el archivo `tests/invalid/errores_simbolos.csc`, que contiene símbolos no permitidos por la gramática de CalcScript: `$` y `@`.

Archivo de entrada — `tests/invalid/errores_simbolos.csc`:

```
x = 10 $ 2;
y = @total + 1;
z = x != y;
print(z);
```

Ejecución y salida:

```
$ python src/main.py tests/invalid/errores_simbolos.csc
```

Token List:

```
1: (IDENTIFIER, "x")
2: (OPERATOR, "=")
3: (NUMBER_INT, "10")
4: (NUMBER_INT, "2")
5: (DELIMITER, ";")
6: (IDENTIFIER, "y")
7: (OPERATOR, "=")
8: (IDENTIFIER, "total")
9: (OPERATOR, "+")
10: (NUMBER_INT, "1")
11: (DELIMITER, ";")
12: (IDENTIFIER, "z")
13: (OPERATOR, "=")
14: (IDENTIFIER, "x")
15: (OPERATOR, "!=")
16: (IDENTIFIER, "y")
17: (DELIMITER, ";")
18: (KEYWORD, "print")
19: (DELIMITER, "(")
20: (IDENTIFIER, "z")
```

```
21: (DELIMITER, ")")
22: (DELIMITER, ";")
```

Error List:

```
Line 1, Column 8: Unknown symbol '$' detected
Line 2, Column 5: Unknown symbol '@' detected
```

Captura 5.2 — El lexer detecta los dos símbolos inválidos, reporta su ubicación exacta y continúa procesando el resto del programa.

5.3 Caso con identificadores inválidos y símbolos desconocidos

Se tokeniza el archivo `tests/invalid/errores_mixtos.csc`, que combina identificadores que comienzan con dígito, un flotante malformado (sin dígitos tras el punto, que el lexer trata como entero + símbolo inválido) y un comentario.

Archivo de entrada — `tests/invalid/errores_mixtos.csc`:

```
123abc = 5;
result = 3. + 2;
x = total # suma parcial
print(x);
```

Nota: # inicia un comentario en CalcScript, por lo que `# suma parcial` es descartado. La línea 3 produce el token `total` y luego el lexer ignora el comentario correctamente.

Ejecución y salida:

```
$ python src/main.py tests/invalid/errores_mixtos.csc
```

Token List:

```
1: (NUMBER_INT, "123")
2: (IDENTIFIER, "abc")
3: (OPERATOR, "=")
4: (NUMBER_INT, "5")
5: (DELIMITER, ";")
6: (IDENTIFIER, "result")
7: (OPERATOR, "=")
8: (NUMBER_INT, "3")
9: (OPERATOR, "+")
10: (NUMBER_INT, "2")
11: (DELIMITER, ";")
12: (IDENTIFIER, "x")
13: (OPERATOR, "=")
14: (IDENTIFIER, "total")
15: (KEYWORD, "print")
16: (DELIMITER, "(")
17: (IDENTIFIER, "x")
18: (DELIMITER, ")")
19: (DELIMITER, ";")
```

Error List:

```
Line 1, Column 1: Invalid identifier: identifiers cannot start with a digit near '1'
Line 2, Column 11: Unknown symbol '.' detected
```

Captura 5.3 — El lexer reconoce correctamente 123 como NUMBER_INT y abc como IDENTIFIER separados; reporta que la secuencia comenzó con dígito. El punto flotante malformado 3. (sin dígitos tras el punto) genera

el error en la posición exacta del punto.

Sección 6 — Conclusiones

Ana Sofía Ramírez Vega. Diseñar la tabla de tokens y las expresiones regulares me permitió entender en la práctica por qué el orden de las reglas es crítico: cuando moví `NUMBER_INT` antes que `NUMBER_FLOAT`, el lexer tokenizaba `3.14` como tres piezas separadas. Verlo fallar y corregirlo fue la forma más efectiva de interiorizar el concepto.

Luis Eduardo Torres Garza. El mayor reto fue el manejo de errores léxicos sin detener el análisis del resto del programa. Implementar el mecanismo de “avanzar un carácter y continuar” requirió pensar cuidadosamente en los estados posibles del lexer; el resultado fue un módulo de errores mucho más útil para el usuario final.

Daniela Hernández Leal. Separar las reglas léxicas (`rules.py`) de la lógica del lexer (`lexer.py`) facilitó enormemente las pruebas: podía modificar una expresión regular y ejecutar inmediatamente los casos de prueba sin tocar el resto del código. Esa separación de responsabilidades es algo que aplicaré en proyectos futuros.

Carlos Iván Morales Peña. Implementar la detección del error de “identificador que comienza con dígito” requirió distinguir entre el caso donde el dígito es parte de un `NUMBER_INT` legítimo y el caso donde le sigue inmediatamente una letra. La solución fue verificar el carácter siguiente al entero reconocido dentro del bucle principal, lo cual me hizo entender mejor los *lookahead* en los lexers.

Trabajo futuro (esta sección totalmente opcional)

- Extender el lexer para reconocer cadenas multilínea con `"""`.
- Agregar soporte para números en notación científica (`1.5e10`).
- Integrar el analizador léxico con un analizador sintáctico (*parser*) para validar también la gramática del lenguaje.
- Generar un reporte HTML coloreado de los tokens por categoría.

Sección 7 — Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson Addison-Wesley.
- [2] Cooper, K. D. y Torczon, L. (2011). *Engineering a Compiler* (2.^a ed.). Morgan Kaufmann.
- [3] Python Software Foundation. (2024). *re — Regular expression operations*. Recuperado de <https://docs.python.org/3/library/re.html>
- [4] Levine, J. (2009). *Flex & Bison: Text Processing Tools*. O'Reilly Media.
- [5] Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
- [6] Python Software Foundation. (2024). *enum — Support for enumerations*. Recuperado de <https://docs.python.org/3/library/enum.html>